

Syntax Analysis (Parsing)

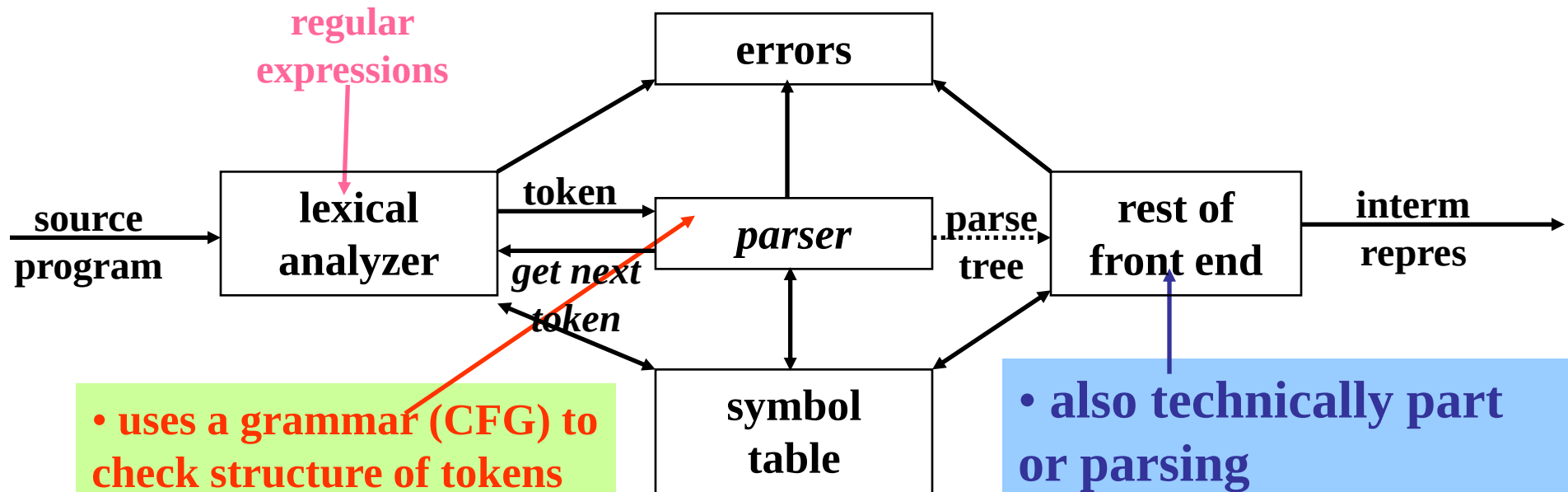
- ◆ An overview of parsing :
 - Functions & Responsibilities
- ◆ Context Free Grammars
 - Concepts & Terminology
- ◆ Writing and Designing Grammars
- ◆ Resolving Grammar Problems / Difficulties
- ◆ Top-Down Parsing
 - Recursive Descent & Predictive LL
- ◆ Bottom-Up Parsing
 - LR & LALR
- ◆ Concluding Remarks/Looking Ahead

Parsers (cont.)

- ◆ We categorize the parsers into two groups:
 1. **Top-Down Parser (自顶向下)**
 - the parse tree is created top to bottom, starting from the root.
 1. **Bottom-Up Parser (自底向上)**
 - the parse tree is created bottom to top; starting from the leaves
- ◆ Both top-down and bottom-up parsers scan the input from left to right (one symbol at a time).
- ◆ Efficient top-down and bottom-up parsers can be implemented only for sub-classes of context-free grammars.
 - **LL for top-down parsing**
 - **LR for bottom-up parsing**

Parsing During Compilation

- ◆ Parser works on a stream of tokens.
- ◆ The smallest item is a token.



- uses a grammar (CFG) to check structure of tokens
- produces a parse tree
- syntactic errors and recovery
- recognize correct syntax
- report errors

Skip to parser

- also technically part of parsing
- includes augmenting info on tokens in source, type checking, semantic analysis

Syntax analyzer



- ◆ *Syntax Analyzer* creates the syntactic structure of the given source program.
- ◆ This syntactic structure is mostly a *parse tree*.
- ◆ *Syntax Analyzer* is also known as *parser*.
- ◆ The syntax of a programming is described by a *context-free grammar (CFG)* (上下文无关文法). We will use **BNF (Backus-Naur Form)** notation in the description of CFGs.
- ◆ The syntax analyzer (parser) checks whether a given source program satisfies the rules implied by a context-free grammar or not.
 - If it satisfies, the parser creates the parse tree of that program.
 - Otherwise the parser gives the error messages.
- ◆ A context-free grammar
 - gives a precise syntactic specification of a programming language.
 - the design of the grammar is an initial phase of the design of a compiler.
 - a grammar can be directly converted into a parser by some tools.

RE vs. CFG (p.172)

◆ Regular Expressions

- Basis of lexical analysis
- Represent regular languages

◆ Context Free Grammars

- Basis of parsing
- Represent language constructs
- Characterize context free languages



EXAMPLE: $(a | b)^* abb$

$A0 \rightarrow aA0 \mid bA0 \mid aA1$

$A1 \rightarrow bA2$

$A2 \rightarrow bA3$

$A3 \rightarrow \epsilon$

Context-Free Grammars

- ◆ Inherently **recursive** structures of a programming language are defined by a context-free grammar.
- ◆ In a context-free grammar, we have:
 - A finite set of **terminals** (终结符) (in our case, this will be the set of tokens)
 - A finite set of **non-terminals** (非终结符) (syntactic-variables)
 - A finite set of **productions rules** (产生式) in the following form
 - $A \rightarrow \alpha$ where A is a non-terminal and α is a string of terminals and non-terminals (including the empty string)
 - A **start symbol** (one of the non-terminal symbol)

◆ Example:

$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid - E$

$E \rightarrow (E)$

$E \rightarrow \text{id}$

$E \rightarrow \text{num}$

Concepts & Terminology

Definition: A **Context Free Grammar (CFG)**, is described by T , NT , S , PR , where:

T: Terminals / tokens of the language

NT: Non-terminals to denote sets of strings generated by the grammar & in the language

S: Start symbol, $S \in NT$, which defines all strings of the language

PR: Production rules to indicate how T and NT are combined to generate valid strings of the language.

$$\mathbf{PR: NT \rightarrow (T \mid NT)^*}$$

Like a Regular Expression / DFA / NFA, a Context Free Grammar is a mathematical model

Example Grammar



$expr \rightarrow expr \ op \ expr$

$expr \rightarrow (\ expr \)$

$expr \rightarrow - \ expr$

$expr \rightarrow id$

$op \rightarrow +$

$op \rightarrow -$

$op \rightarrow *$

$op \rightarrow /$

$op \rightarrow \uparrow$

Black : NT

Blue : T

$expr : S$

9 Production rules

To simplify / standardize notation, we offer a synopsis of terminology.

Terminology (cont.)

- ◆ $L(G)$ is *the language of G* (the language generated by G) which is a set of sentences.
- ◆ A **sentence** (句子) of $L(G)$ is a string of terminal symbols of G .
- ◆ If S is the start symbol of G then
 ω is a sentence of $L(G)$ if $S \Rightarrow^+ \omega$ where ω is a string of terminals of G .
- ◆ A language that can be generated by a grammar is said to be a **context-free language**.
- ◆ If G is a context-free grammar, $L(G)$ is a *context-free language*.
- ◆ Two grammars are *equivalent* if they produce the same language.
 $S \xRightarrow{*} \alpha$
 - If α contains non-terminals, it is called as a **sentential form** of G .
 - If α does not contain non-terminals, it is called as a **sentence** of G .

EX. $E \Rightarrow E+E \Rightarrow \text{id}+E \Rightarrow \text{id}+\text{id}$

How does this relate to Languages?

Let G be a CFG with start symbol S . Then $S \Rightarrow^+ W$ (where W has no non-terminals) represents the language generated by G , denoted $L(G)$. So

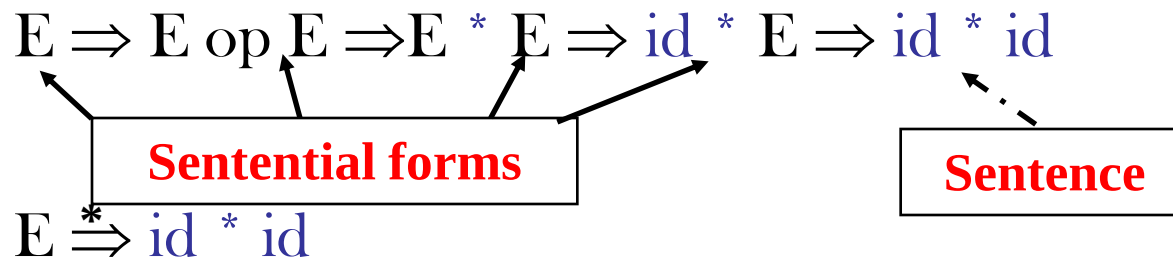
$$W \in L(G) \Leftrightarrow S \Rightarrow^+ W.$$

W : is a sentence of G

When $S \Rightarrow^* \alpha$ (and α may have NTs) it is called a **sentential form of G** .

EXAMPLE: $id * id$ is a sentence

Here's the derivation:



Example Grammar – Terminology

Terminals: $a, b, c, +, -, \text{punc}, 0, 1, \dots, 9$, blue strings

Non Terminals: A, B, C, S , black strings

T or NT: X, Y, Z

Strings of Terminals: $u, v, \dots, z$ in T^*

Strings of T / NT: α, β, γ in $(T \cup NT)^*$

Alternatives of production rules:

$$A \rightarrow \alpha_1; A \rightarrow \alpha_2; \dots; A \rightarrow \alpha_k; \Rightarrow A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_k$$

First NT on LHS of 1st production rule is designated as start symbol !

$$E \rightarrow E \text{ op } E \mid (E) \mid -E \mid \text{id}$$

$$\text{op} \rightarrow + \mid - \mid * \mid / \mid \uparrow$$

Derivations

The central idea here is that a production is treated as a **rewriting rule** (重写规则) in which the non-terminal on the left is replaced by the string on the right side of the production.

$E \Rightarrow E+E$

- $E+E$ derives from E
 - we can replace E by $E+E$
 - to able to do this, we have to have a production rule $E \rightarrow E+E$ in our grammar.

$E \Rightarrow E+E \Rightarrow id+E \Rightarrow id+id$

- A sequence of replacements of non-terminal symbols is called a **derivation** (推导) of $id+id$ from E .
- In general a derivation step is

$\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$ (α_n derives from α_1 or α_1 derives α_n)

Grammar Concepts

A step in a derivation is zero or one action that replaces a NT with the RHS of a production rule.

EXAMPLE: $E \Rightarrow -E$ (the $\Rightarrow$ means “derives” in one step) using the production rule: $E \rightarrow -E$

EXAMPLE: $E \Rightarrow E \text{ op } E \Rightarrow E * E \Rightarrow E * (E)$

DEFINITION:

- $\Rightarrow$ derives in one step
- $\Rightarrow^+$ derives in $\geq$ one step
- $\Rightarrow^*$ derives in $\geq$ zero steps

EXAMPLES: $\alpha A \beta \Rightarrow \alpha \gamma \beta$ if $A \rightarrow \gamma$ is a production rule

$\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$ then $\alpha_1 \Rightarrow^* \alpha_n$; $\alpha \Rightarrow^* \alpha$ for all α

If $\alpha \Rightarrow^* \beta$ and $\beta \rightarrow \gamma$ then $\alpha \Rightarrow^* \gamma$

Other Derivation Concepts

Example 4.4 (ref. Example (4.3))

OR $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(id+E) \Rightarrow -(id+id)$

$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(E+id) \Rightarrow -(id+id)$

- ◆ At each derivation step, we can choose any of the non-terminal in the sentential form of G for the replacement.
- ◆ If we always choose the left-most non-terminal in each derivation step, this derivation is called as **left-most derivation** (最左推导) .
- ◆ If we always choose the right-most non-terminal in each derivation step, this derivation is called as **right-most derivation** (最右推导) .

Derivation Example

Leftmost: Replace the leftmost non-terminal symbol

$$E \Rightarrow_{\text{lm}} E \text{ op } E \Rightarrow_{\text{lm}} \text{id op } E \Rightarrow_{\text{lm}} \text{id} * E \Rightarrow_{\text{lm}} \text{id} * \text{id}$$

Rightmost: Replace the leftmost non-terminal symbol

$$E \Rightarrow_{\text{rm}} E \text{ op } E \Rightarrow_{\text{rm}} E \text{ op } \text{id} \Rightarrow_{\text{rm}} E * \text{id} \Rightarrow_{\text{rm}} \text{id} * \text{id}$$

Important Notes: $A \rightarrow \delta$

$$\text{If } \beta A \gamma \Rightarrow_{\text{lm}} \beta \delta \gamma$$

$$\text{If } \beta A \gamma \Rightarrow_{\text{rm}} \beta \delta \gamma$$

what's true about ~~β~~ ?

Derivations: Actions to parse input can be represented pictorially in a parse tree.

Left-Most and Right-Most Derivations

Left-Most Derivation

$$\begin{array}{ccccccc} E & \Rightarrow & -E & \Rightarrow & -(E) & \Rightarrow & -(E+E) & \Rightarrow & -(id+E) & \Rightarrow & -(id+id) \\ \text{lm} & & \text{lm} & & \text{lm} & & \text{lm} & & \text{lm} & & \end{array} \quad (4.4)$$

Right-Most Derivation (called *canonical derivation*)

$$\begin{array}{ccccccc} E & \Rightarrow & -E & \Rightarrow & -(E) & \Rightarrow & -(E+E) & \Rightarrow & -(E+id) & \Rightarrow & -(id+id) \\ \text{rm} & & \text{rm} & & \text{rm} & & \text{rm} & & \text{rm} & & \end{array}$$

- We will see that the *top-down parsers* try to find the *left-most derivation* of the given source program.

How mapping?

- We will see that the *bottom-up parsers* try to find the *right-most derivation* of the given source program in the reverse order.

Derivation exercise 1 in class

Productions:

$assign_stmt \rightarrow id := expr ;$

$expr \rightarrow expr \ op \ term$

$expr \rightarrow term$

$term \rightarrow id$

$term \rightarrow real$

$term \rightarrow integer$

$op \rightarrow +$

$op \rightarrow -$

Let's derive:

$id := id + real - integer ;$

Please use left-most derivation or right-most derivation.

Let's derive: $id := id + real - integer;$

Left-most derivation:

$assign_stmt$

$\rightarrow id := expr;$

$\rightarrow id := expr \ op \ term;$

$\rightarrow id := expr \ op \ term \ op \ term;$

$\rightarrow id := term \ op \ term \ op \ term;$

$\rightarrow id := id \ op \ term \ op \ term;$

$\rightarrow id := id + term \ op \ term;$

$\rightarrow id := id + real \ op \ term;$

$\rightarrow id := id + real - term;$

$\rightarrow id := id + real - integer;$

using production:

$assign_stmt \rightarrow id := expr;$

$expr \rightarrow expr \ op \ term$

$expr \rightarrow expr \ op \ term$

$expr \rightarrow term$

$term \rightarrow id$

$op \rightarrow +$

$term \rightarrow real$

$op \rightarrow -$

$term \rightarrow integer$

Parse Tree

- Inner nodes of a parse tree are non-terminal symbols.
- The leaves of a parse tree are terminal symbols.

A **parse tree** can be seen as a graphical representation of a derivation.

EX. $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(id+E) \Rightarrow -(id+id)$ (4.4)

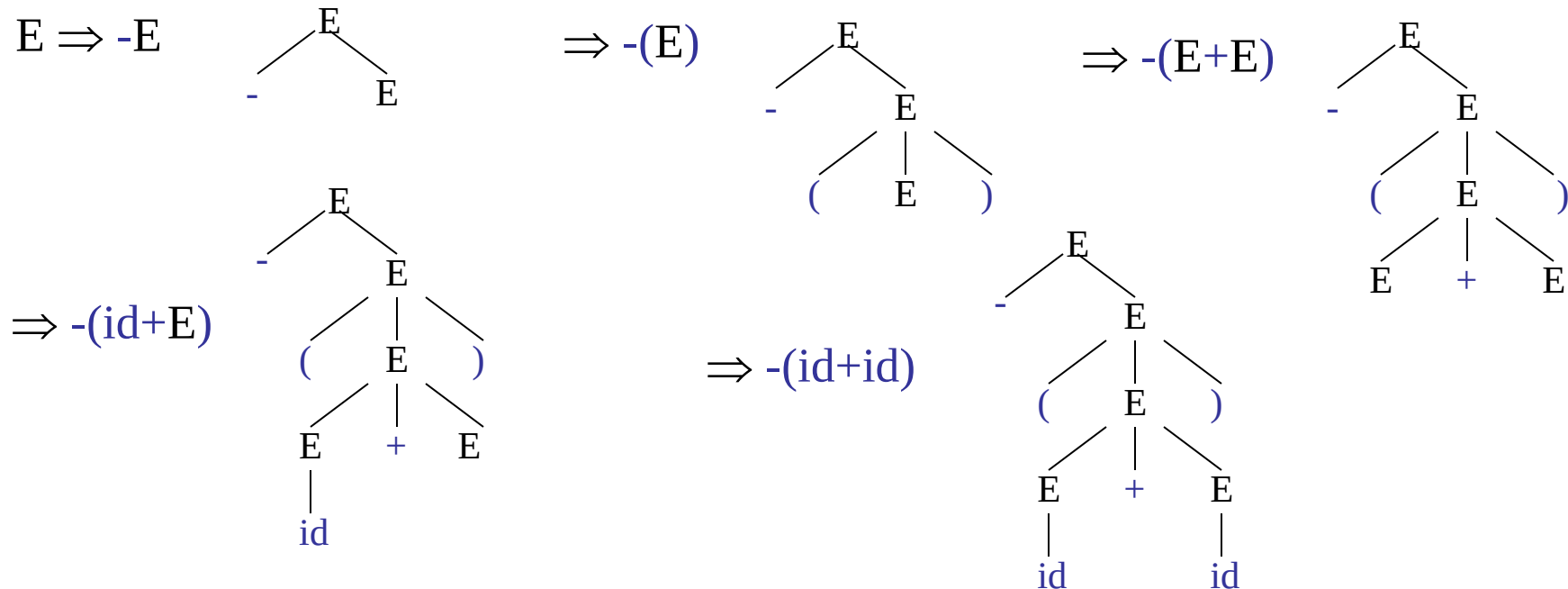


Fig. 4.3 building the parse tree from derivation (4.4)

Examples of LM / RM Derivations

$$E \rightarrow E \text{ op } E \mid (E) \mid -E \mid \text{id}$$
$$\text{op} \rightarrow + \mid - \mid * \mid / \mid \uparrow$$

A leftmost derivation of : $\text{id} + \text{id} * \text{id}$

[See latter slides](#)

A rightmost derivation of : $\text{id} + \text{id} * \text{id}$

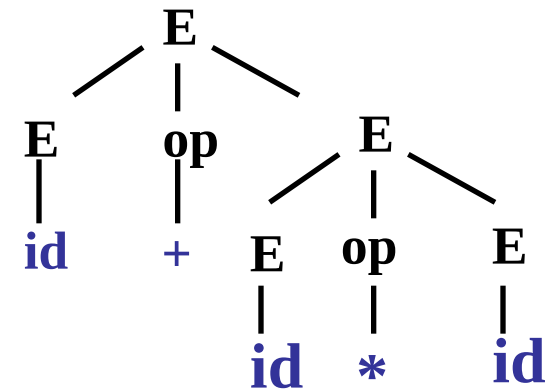
DO IT BY YOURSELF.

$E \Rightarrow E \text{ op } E$

$\Rightarrow \text{id op } E$

$\Rightarrow \text{id} + E$

$\Rightarrow \text{id} + E \text{ op } E$



$\Rightarrow \text{id} + \text{id op } E$

$\Rightarrow \text{id} + \text{id} * E$

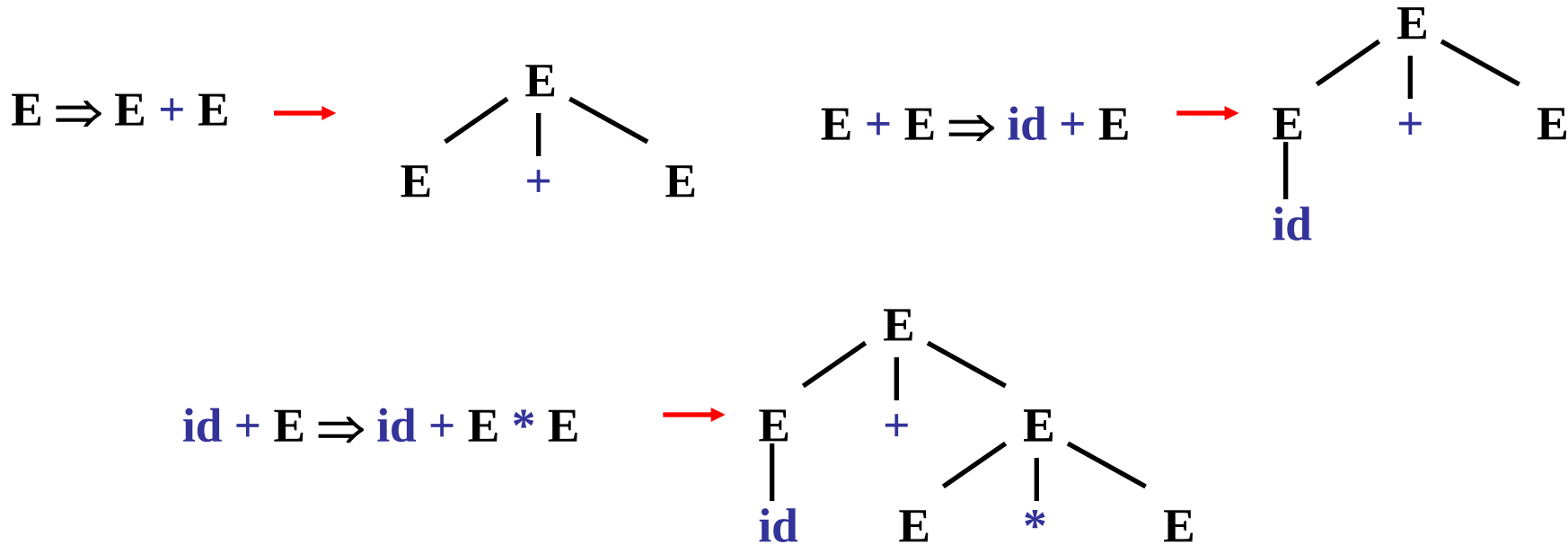
$\Rightarrow \text{id} + \text{id} * \text{id}$

Parse Trees and Derivations

Consider the expression grammar:

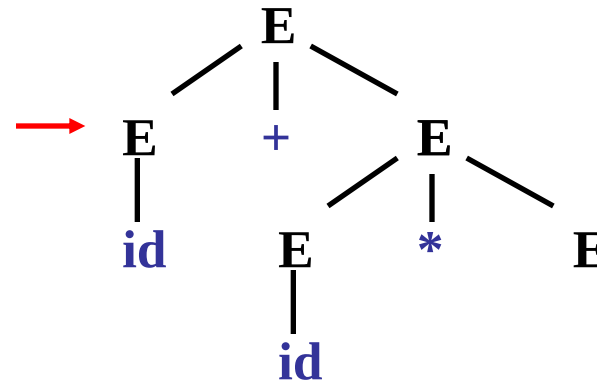
$$E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$$

Leftmost derivations of $\text{id} + \text{id} * \text{id}$

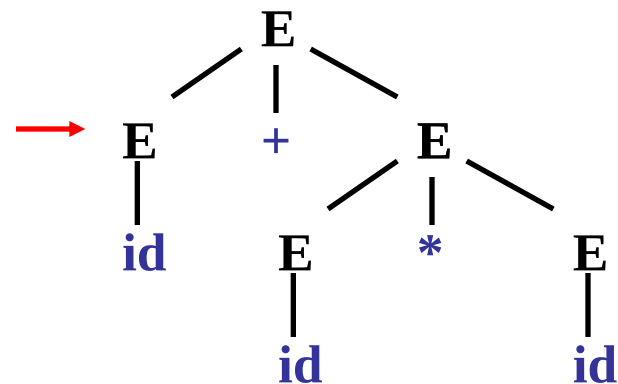


Parse Tree & Derivations (cont.)

$\text{id} + \text{E} * \text{E} \Rightarrow \text{id} + \text{id} * \text{E}$



$\text{id} + \text{id} * \text{E} \Rightarrow \text{id} + \text{id} * \text{id}$



Alternative Parse Tree & Derivation

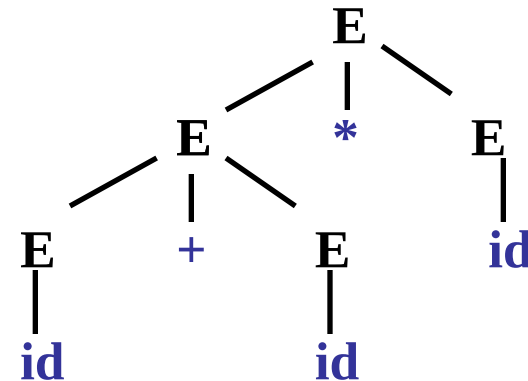
$E \Rightarrow E * E$

$\Rightarrow E + E * E$

$\Rightarrow id + E * E$

$\Rightarrow id + id * E$

$\Rightarrow id + id * id$



WHAT'S THE ISSUE HERE ?

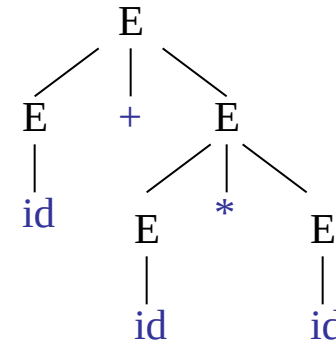
Two distinct leftmost derivations!

Ambiguity (二义性)

- A grammar produces more than one parse tree for a sentence is called as an *ambiguous* grammar.

Example 4.6:

$$\begin{aligned} E &\Rightarrow E + E \Rightarrow \text{id} + E \Rightarrow \text{id} + E * E \\ &\Rightarrow \text{id} + \text{id} * E \Rightarrow \text{id} + \text{id} * \text{id} \end{aligned}$$



$$\begin{aligned} E &\Rightarrow E * E \Rightarrow E + E * E \Rightarrow \text{id} + E * E \\ &\Rightarrow \text{id} + \text{id} * E \Rightarrow \text{id} + \text{id} * \text{id} \end{aligned}$$

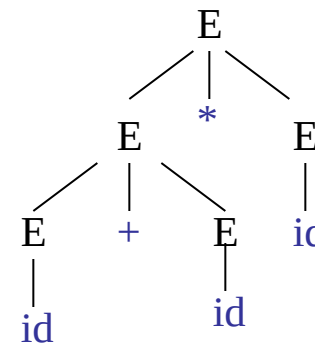


Fig. 4.4 two parse trees for $\text{id} + \text{id} * \text{id}$.

Ambiguity (cont.)

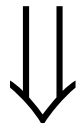
- For the most parsers, the grammar must be unambiguous.
- unambiguous grammar
 - ➔ unique selection of the parse tree for a sentence
- We should eliminate the ambiguity in the grammar during the design phase of the compiler.
- An ambiguous grammar should be written to eliminate the ambiguity.
- We have to prefer one of the parse trees of a sentence (generated by an ambiguous grammar) to disambiguate that grammar to restrict to this choice by “throw away” undesirable parse trees.

Ambiguity – Operator Precedence

- Ambiguous grammars (because of ambiguous operators) can be disambiguated according to the **precedence** and **associativity** rules.

$$E \rightarrow E + E \mid E * E \mid E \wedge E \mid \text{id} \mid (E)$$

disambiguate the grammar



precedence:



$\wedge$ (right to left)
 $*$ (left to right)
 $+$ (left to right)

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow G \wedge F \mid G$$

$$G \rightarrow \text{id} \mid (E)$$

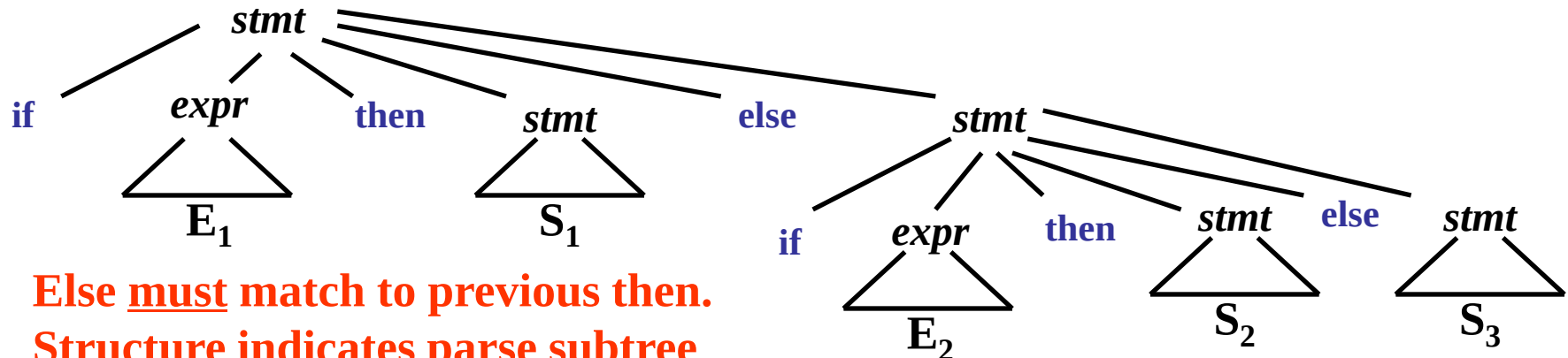
Eliminating Ambiguity (P.174)

Consider the following grammar segment:

$stmt \rightarrow$ **if** *expr* **then** *stmt*
 | **if** *expr* **then** *stmt* **else** *stmt*
 | **other** (any other statement)

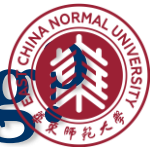
What's problem here ?

Let's consider a simple parse tree:



Else must match to previous then.
Structure indicates parse subtree
for expression.

Example : What Happens with this string



if E_1 then if E_2 then S_1 else S_2

How is this parsed ?

if E_1 then
 if E_2 then
 S_1
else
 S_2

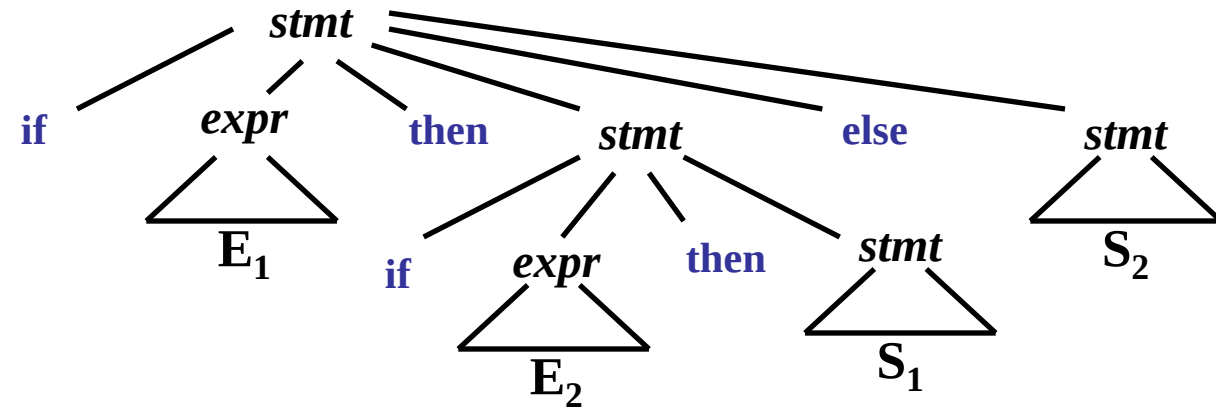
vs.

if E_1 then
 if E_2 then
 S_1
else
 S_2

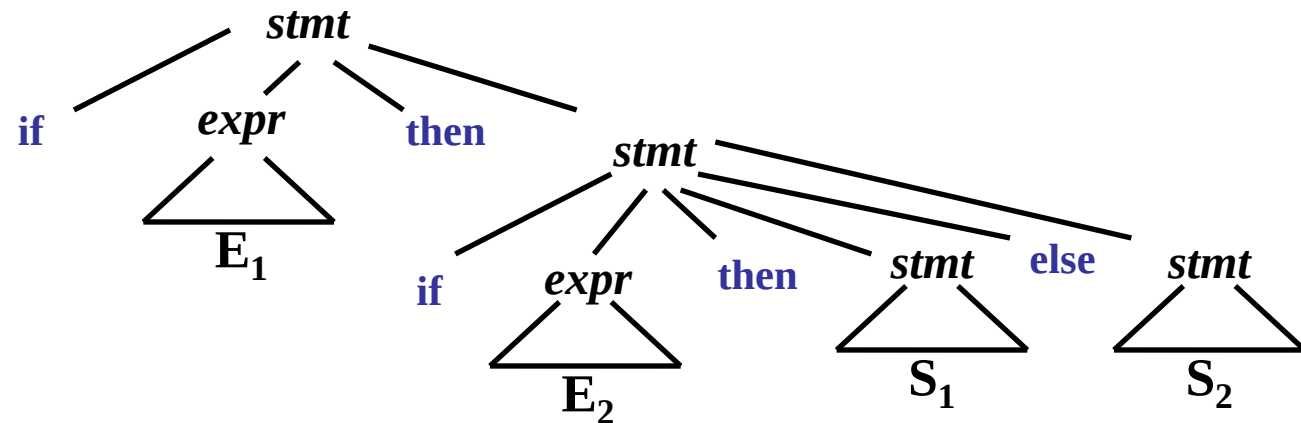
What's the issue here ?

Parse Trees for Example

Form 1:



Form 2:



What's the issue here ?

Fig. 4.6 two parse trees for an ambiguous sentence.

Ambiguity (cont.)

- We prefer the second parse tree (else matches with closest if).
- So, we have to disambiguate our grammar to reflect this choice.
- The unambiguous grammar will be:

$$\begin{aligned} \text{stmt} &\rightarrow \text{matchedstmt} \\ &\quad | \text{unmatchedstmt} \\ \text{matchedstmt} &\rightarrow \text{if expr then matchedstmt else matchedstmt} \\ &\quad | \text{otherstmts} \\ \text{unmatchedstmt} &\rightarrow \text{if expr then stmt} \\ &\quad | \text{if expr then matchedstmt else unmatchedstmt} \end{aligned} \tag{4.9}$$

The general rule is “match each **else** with the closest previous unmatched **then**.”

Left Recursion (P.176)

- A grammar is *left recursive* (左递归) if it has a non-terminal A such that there is a derivation $A \Rightarrow A\alpha^+$ for some string α .
- The left-recursion may appear in a single step of the derivation (*immediate left-recursion* 直接左递归), or may appear in more than one step of the derivation.
- Top-down parsing techniques cannot handle left-recursive grammars.
why?
- So, we have to convert our left-recursive grammar into an equivalent grammar which is not left-recursive.

Why is Left Recursion a Problem ?

Consider:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Derive : $id + id + id$

$$E \Rightarrow E + T \Rightarrow$$

How can left recursion be removed ?

$$E \rightarrow E + T \mid T$$

What does this generate?

$$E \Rightarrow E + T \Rightarrow T + T$$

$$E \Rightarrow E + T \Rightarrow E + T + T \Rightarrow T + T + T$$

...

How does this build strings ?

What does each string have to start with ?

Immediate Left-Recursion

$A \rightarrow A \alpha \mid \beta$ where β does not start with A

$\Downarrow$

eliminate immediate left recursion

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \varepsilon$ an equivalent grammar: replaced by **right-recursion**

In general,

$A \rightarrow A \alpha_1 \mid \dots \mid A \alpha_m \mid \beta_1 \mid \dots \mid \beta_n$ where $\beta_1 \dots \beta_n$ do not start with A

$\Downarrow$

eliminate immediate left recursion

$A \rightarrow \beta_1 A' \mid \dots \mid \beta_n A'$

$A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_m A' \mid \varepsilon$ an equivalent grammar

Eliminate immediate left recursion exercise 1 in class



Example 4.8:

$$E \rightarrow E+T \mid T$$

$$T \rightarrow T*F \mid F$$

$$F \rightarrow \text{id} \mid (E) \quad (4.10)$$



$$E \rightarrow T E'$$

$$E' \rightarrow +T E' \mid \varepsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow *F T' \mid \varepsilon$$

$$F \rightarrow \text{id} \mid (E) \quad (4.11)$$

Left-Recursion – Problem

- ◆ A grammar cannot be immediately left-recursive, but it still can be left-recursive.
- ◆ By just eliminating the immediate left-recursion, we may not get a grammar which is not left-recursive.

$$S \rightarrow Aa \mid b$$
$$A \rightarrow Sc \mid d$$

This grammar is not immediately left-recursive,
but it is still left-recursive.

$$\underline{S} \Rightarrow Aa \Rightarrow \underline{S}ca$$

or $\underline{A} \Rightarrow Sc \Rightarrow \underline{A}ac$

causes to a left-recursion

- So, we have to eliminate all left-recursions from our grammar

Elimination of Left-Recursion (P.176)

Algorithm 4.1. eliminating left recursion.

- Arrange non-terminals in some order: $A_1 \dots A_n$
- for i from 1 to n do {
 - for j from 1 to $i-1$ do {
 - replace each production
$$A_i \rightarrow A_j \gamma$$
by
$$A_i \rightarrow \alpha_1 \gamma \mid \dots \mid \alpha_k \gamma$$
where $A_j \rightarrow \alpha_1 \mid \dots \mid \alpha_k$ are all the current A_j -productions.
- eliminate immediate left-recursions among A_i -productions

Fig. 4.7 algorithm to eliminate left recursion from a grammar.

Example

Example 4.9:

$S \rightarrow Aa \mid b$

$A \rightarrow Ac \mid Sd \mid \varepsilon$

(4.12)

- Order of non-terminals: S, A

for S:

- we do not enter the inner loop.
- there is no immediate left recursion in S.

for A:

- **Replace $A \rightarrow Sd$ with $A \rightarrow Aad \mid bd$**
So, we will have $A \rightarrow Ac \mid Aad \mid bd \mid \varepsilon$
- Eliminate the immediate left-recursion in A

$$\begin{aligned} A &\rightarrow bdA' \mid A' \\ A' &\rightarrow cA' \mid adA' \mid \varepsilon \end{aligned}$$

So, the resulting equivalent grammar which is not left-recursive is:

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow bdA' \mid A' \\ A' &\rightarrow cA' \mid adA' \mid \varepsilon \end{aligned}$$

not immediately left-recursive



immediate left-recursive



eliminating immediate
left-recursive

Left-Factoring (左因式分解)

- A predictive parser (a top-down parser without backtracking) insists that the grammar must be *left-factored*. The basic idea is that when it is not clear which of two alternative productions to use to expand a nonterminal A, we may be able to rewrite the A-productions to defer the decision until we have seen enough of the input to make the right choice.

grammar $\rightarrow$ a new equivalent grammar suitable for predictive parsing

stmt $\rightarrow$ if expr then stmt else stmt
| if expr then stmt

When shift and reduce conflict
in YACC, shift is first.

- when we see if, we cannot know which production rule to choose to re-write *stmt* in the derivation.

Left-Factoring (cont.)

Problem : Uncertain which of 2 rules to choose:

$$\begin{aligned} stmt &\rightarrow \text{if } expr \text{ then } stmt \text{ else } stmt \\ &| \text{if } expr \text{ then } stmt \end{aligned}$$

When do you know which one is valid ?

What's the general form of *stmt* ?

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$$

$$\begin{aligned} \alpha &: \text{if } expr \text{ then } stmt \\ \beta_1 &: \text{else } stmt \quad \beta_2 : \epsilon \end{aligned}$$

Transform to:

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 \mid \beta_2$$

EXAMPLE:

$$stmt \rightarrow \text{if } expr \text{ then } stmt \text{ rest}$$

$$rest \rightarrow \text{else } stmt \mid \epsilon$$

so, we can immediately expand *A* to $\alpha A'$

Left-Factoring – Algorithm

For each non-terminal A with two or more alternatives (production rules) with a common non-empty prefix, let say

$$A \rightarrow \alpha\beta_1 \mid \dots \mid \alpha\beta_n \mid \gamma_1 \mid \dots \mid \gamma_m$$

convert it into

$$A \rightarrow \alpha A' \mid \gamma_1 \mid \dots \mid \gamma_m$$

$$A' \rightarrow \beta_1 \mid \dots \mid \beta_n$$

Left-Factoring – Example1

$$A \rightarrow \underline{a}bB \mid \underline{a}B \mid cdg \mid cdeB \mid cdfB$$

$\Downarrow$ step1

$$A \rightarrow aA' \mid \underline{cdg} \mid \underline{cde}B \mid \underline{cdf}B$$
$$A' \rightarrow bB \mid B$$

$\Downarrow$ step 2

$$A \rightarrow aA' \mid cdA''$$
$$A' \rightarrow bB \mid B$$
$$A'' \rightarrow g \mid eB \mid fB$$

Left-Factoring exercise 1 in class

$$A \rightarrow ad \mid a \mid ab \mid abc \mid b$$

$$A \rightarrow ad \mid a \mid ab \mid abc \mid b$$

↓ step 1

$$A \rightarrow aA' \mid b$$

$$A' \rightarrow d \mid \varepsilon \mid b \mid bc$$

↓ step 2

$$A \rightarrow aA' \mid b$$

$$A' \rightarrow d \mid \varepsilon \mid bA''$$

$$A'' \rightarrow \varepsilon \mid c$$

Normal form:

$$A \rightarrow aA' \mid b$$

$$A' \rightarrow bA'' \mid d \mid \varepsilon$$

$$A'' \rightarrow c \mid \varepsilon$$

Non-Context Free Language Constructs

- There are some language constructions in the programming languages which are not context-free. This means that, we cannot write a context-free grammar for these constructions.

Example 4.11:

- $L1 = \{ \omega c \omega \mid \omega \text{ is in } (a \mid b)^* \}$ is not context-free
 - ➔ declaring an identifier and checking whether it is declared or not later. We cannot do this with a context-free language. We need semantic analyzer (which is not context-free).

Example 4.12:

- $L2 = \{ a^n b^m c^n d^m \mid n \geq 1 \text{ and } m \geq 1 \}$ is not context-free
 - ➔ declaring two functions (one with n parameters, the other one with m parameters), and then calling them with actual parameters.

小结

- ◆理解上下文无关文法、句型、句子的概念，掌握最左推导、最右推导及语法树的构建
- ◆理解文法的二义性概念以及如何消除文法的二义性
- ◆掌握左递归消除和左因式分解的方法